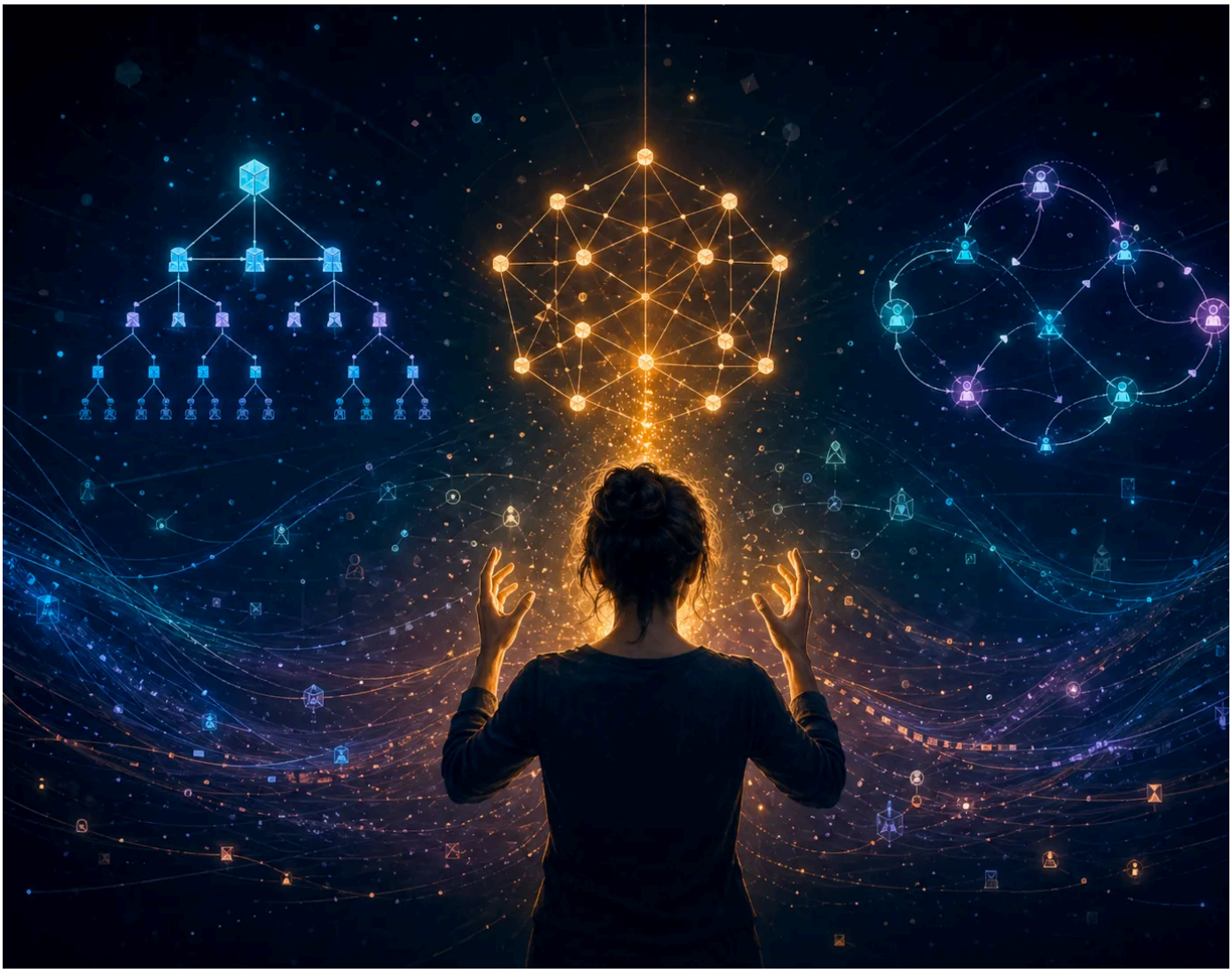
Configurations of Mind

Matching orchestration frameworks to cognitive architecture

ERIC KOZIOL

APR 28, 2026

5 1 1 Sh



"The structure of the organization determines what it can accomplish."

-Peter Drucker

In the past six months, I have watched many multi-agent orchestration frameworks appear. Gastown, Metaswarm, gStack, OpenClaw, OpenFang, Citadel, Ruflo, Claude Squad, and Conductor to name a few. People are building and shipping these week What struck me is not the volume. It is that the people getting the best results from these systems are rarely using the same one. **When the right mind meets the rigl framework, large amplifications happen.** Most people are nowhere near that experience because they have not done the matching work.

One of my colleagues didn't like Gastown. Didn't like gStack either. So they built the own orchestration framework modeled on NATO's command-and-decision structur for defense operations. And it works better for them than anything else on the mark **That's actually the whole point.** A framework that makes one person superhuman can leave another spinning. The variable that explains the difference is the operator mind.

The Tool Belt Thesis, Revisited

In my predictions for the year, I wrote about how tool belts (load outs) will become prevalent and based on how you think. That prediction is materializing faster than expected. Since my article on Gastown came out, the pace has only accelerated. Ea new framework takes a different angle on two questions: how do I create a virtual worker, and how do I create a superhuman worker?

These are fundamentally different pursuits. One is about replacing human labor. Th other is about amplifying it. I have been spending my time on the amplification side because the dynamics are wilder than most people realize. Every framework has a capability ceiling. Every operator has a realized output. The gap between those two cognitive fit.

The gap between a framework's capability and your realized output is determined by how well the tool fits your cognitive architecture. I have watche dozens of people work with these systems. When the fit is right, the user and the framework coalesce. When it is wrong, you spend more time fighting the tool than doing the work. The people getting outsized results are not using the "best" framework. They are using the one that maps to how they already think.

The Sapir-Whorf Hypothesis for AI Tooling

There is a principle from linguistics that maps cleanly onto this problem. The Sapir-Whorf hypothesis proposes that the language you speak shapes how you think anc what you can express. The strong version claims language determines thought. Th weak version says language influences thought.

If a language merely influences how its speakers think, that influence compounds across every problem they frame and every solution they reach. Orchestration frameworks function as languages too. The system you choose to structure AI worl becomes the grammar through which you express intent, decompose problems, ar evaluate results. Pick a grammar that fits how you already think, and you spend you energy on the work. Pick one that does not, and you spend it on translation.

The system you use to organize AI work shapes the types of tasks you accomplish and how you accomplish them. A military-style command hierarchy channels you toward different problem decompositions than a flat, consensus-driv model. Neither is wrong. They produce different outcomes because they structure thought differently.

Give an empathetic person a military orchestration framework and they will fight it. Give a military-minded person that same framework and it amplifies everything the are good at. The framework does not create capability. It amplifies what is already

there. A military system may be less effective at economic influence than an empathetic framework optimized for relationship-building.

This is the personalized software thesis pushed further. We are past "hey, build the tools." We are at **"build the right tool for the right mind."** We can now fit the software to the individual's process instead of forcing the individual to fit into the software.

Finding Your Framework

You have to experiment. Build your own framework. Augment an existing one. Combine a few together. Nobody has mapped which configurations produce which results for which types of people. That knowledge is forming, but right now the onl way to find your fit is to try things and measure what happens.

Borrow from people at the edge who are finding configurations that work for them. Test their discoveries against your own cognitive style. The Bruce Lee approach: absorb what is useful, discard what is not, add what is uniquely your own.

However, **do not measure your success in token usage. That metric serves foundation model companies, not you.** Are you shipping faster? Creating things could not create before? Taking on work that was previously too complex? Or are y watching agents spin without producing results? If they are spinning without shippi the framework is not working for your mind. Find another one.

Anatomy of an Orchestration System

Building the right tool for the right mind requires understanding what these tools ar made of. Strip away the branding and every orchestration system is the same set o concepts configured differently:

1. **Role specification.** How agents are defined: system prompts, personas, skill s specialization depth. Some frameworks let you deeply customize each agent's identity and expertise. Others give you fixed agent types. The shape of the wor determines the shape of the work.
2. **Work decomposition and delegation.** How the system breaks a big task into pieces and assigns them. Some use a hierarchy of roles with oversight. Other systems use flat task queues.
3. **Execution.** How work is actually performed: sequential, parallel, hybrid, or rout through hierarchies.
4. **Command flow and routing.** How instructions move through the system: top-down, distributed, or event-driven.
5. **Coordination and work combination.** How agents communicate, synchronize and merge their outputs into a coherent whole. Too much coordination creates bottlenecks. Too little creates drift. The merge problem is often the hardest par and the least discussed.
6. **Review and steering.** How work quality is verified and course-corrected. Som systems have dedicated review agents. Others rely on the human operator to catch problems.
7. **Tooling and capabilities.** What tools each agent and system have access to a how that access is scoped per role. File system, code execution, web browsing MCP servers, external APIs. The tool inventory defines the ceiling of what the system can accomplish.
8. **Interface.** How the operator interacts with the system and how much guidanc needs at various steps. CLI, chat, visual dashboard, code-as-config. The best systems minimize hand-holding without sacrificing control.
9. **Observability.** How you see what is happening while it is happening. Logs, trac dashboards, token burn rates, agent status. Some frameworks are black boxes

until they finish. Others give you a live view. This is the infrastructure that make steering possible.

10. **Artifact management.** How outputs are generated, stored, and surfaced for human review. Can you actually see what the system produced without digging through logs?
11. **State and memory.** How the system persists context across tasks, sessions, a agents. Ephemeral or durable. Shared memory or isolated agents. Can you picl where you left off?
12. **Goals and acceptance criteria.** How the system knows what success looks lil and when a task is complete.
13. **Access and security.** What commands are allowed, what the permission boundaries are, and who can touch what.

Every framework arranges these concepts into a different directed acyclic graph (DAG) with different tools at each node. **The DAG matters more than any individu concept.**

Gastown is strong on decomposition, delegation, and funneling information. It gives you structure and oversight. For raw task execution, frameworks like Ralph Loops a Metaswarm can outperform it. They are more tactical, more focused, better at grinding through a task to completion. Knowing where each framework excels lets pick the right one for the job, or combine them.

Type your email... Subscribe

Testing Organizational Designs Digitally

Every orchestration framework is an organizational design expressed as code. A hierarchy of specialized agents with a reviewer at the top is a management structu

A flat pool of generalists pulling from a shared task queue is a different managemer structure. Same work, different org chart, different results.

People have been arguing about which organizational structures work best for centuries. Entire careers built on the question. The answer has always been "it depends," and the cost of finding out what it depends on was enormous: build actu teams, run them for months, measure what you can, hope your sample size means anything.

That constraint is gone. You can spin up a virtual organization in minutes, point it at real task, and measure what comes out. Run the same task through five different configurations. Compare time, cost, quality, how many times you had to intervene. Change one variable at a time. This is A/B testing for organizational design, and the iteration cycle is hours instead of quarters.

A meta layer will form on top of all orchestration systems. Which configurations produce which results for which types of tasks and which types of people? That is systematic, testable knowledge. **We can now treat organizational design as an empirical question rather than a theoretical debate.** The configuration maps to result and we can study that mapping.

Evaluating Orchestration Systems

Treating organizational design as empirical means we need real measurement. The Intelligence per Watt framework gives us a basis for evaluating task completion, bu captures only one dimension. A framework that scores well on a feature implementation task may collapse on a research synthesis task. You need to measu across the full surface:

Task qualification. Not every framework is well suited for every type of work. Clas work before measuring against it.

- Does a debugging task reward diagnostic reasoning and information funneling
- Does a brainstorming task reward open-ended exploration?
- Does a multi-step build reward structured delegation and review?

Efficiency. How much does the system burn to produce a unit of output? A system that produces great output but burns ten times the tokens is not efficient; it is expensive.

- Tokens, compute, and time per unit of output
- How many iterations before the system converges
- Time from initial prompt to shipped result
- Length of required output; many outputs are better in a smaller form than large one

Effectiveness. Does the system produce work that meets the bar, or does it produ volume that looks like work?

- Quality of task completion against acceptance criteria
- Quality of reasoning: genuine insight or verbose padding?
- Error rates and the total cost of an error when things go wrong

Reliability. Does the system produce consistent results across repeated runs? A framework that scores 90% once and 40% the next time is not a 90% framework. Single-run benchmarks hide this variance.

- Consistency of output quality across multiple runs of the same task
- Variance in token usage and time across identical prompts
- Failure mode predictability: does it fail the same way or differently each time?

Human experience. The best framework on paper is worthless if supervising it tak more effort than doing the work yourself.

- How often do you need to step in?
- Can you see the system's state without digging?
- Can you course-correct mid-flight without restarting everything?
- What happens when an agent fails or goes off-track?

If you are not measuring outcomes, you are guessing.

Where This Is Heading

The opening question was about amplification: what happens when the right mind meets the right framework? The answer is that most people will never find out, because they treat framework selection as a technology decision instead of a cognitive one. They pick the tool with the most stars on GitHub and wonder why th are fighting it.

You now have the vocabulary to do the matching work. Thirteen concepts. Five evaluation dimensions. A way to A/B test organizational designs in hours instead of quarters. The frameworks are all built from similar ingredients. The configuration is what differs, and the configuration that matters is the one that maps to how you already think.

The logical next step is a system where you describe how you think and what you need to accomplish, and the orchestration layer configures itself to match. We are there yet. Until then, the people who invest time in finding their cognitive fit will outperform the people who do not. That gap will only widen.

Subscribe to Embracing Enigmas

By Eric Koziol · Launched 3 years ago

Lessons from applying AI/ML to business from a battle hardened practitioner with an eye towards th future. Read by top venture capitalists, founders, corporate executives, and machine learning practitioners.

Type your email... Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.

---

5 Likes · 1 Restack

Previous

Discussion about this post

Comments Restacks

Write a comment...

Jay Fly 17h

this is amazing because I have been seriously having the same revelations and epiphany's as over the last 12 months, and I have been doing what I call STACKING capability patterns whicl be observed in public domain and refined applied to the mental model and memory anchors (they acronym, visual / image / icon, something that will warp you back to the memory when yu need it). thank you for writing

LIKE REPLY

© 2026 Eric Koziol · Privacy · Terms · Collection notice
Substack is the home for great culture